Synopsis

Network Intrusion Detection for Cyber Security

Problem Id: 9

Student Name: Avni Srivastava

University Roll number: 2028683

Section: P

Student Name: Jigyasa Rana

University Roll number: 2028439

Section: P

- ## Problem Description:

Network intrusion detection is the process of identifying unauthorized or malicious activities occurring within a computer network. With the rapid growth of cyber threats such as hacking attempts, denial-of-service attacks, malware transmission, and data breaches, ensuring network security has become increasingly important. Manual monitoring of large volumes of network traffic data is inefficient and prone to errors.

This project aims to develop a Machine Learning-based Network Intrusion Detection System using a raw, real-world-like dataset containing multiple network traffic features such as IP Address, MAC Address, Protocol Type, Packet Size, Connection Duration, Bytes Sent, Bytes Received, Traffic Flags, and Error Rate. The dataset intentionally includes missing values, variations, and raw attributes to perform preprocessing tasks like data cleaning, handling null values, feature analysis, and visualization before model training.

In real-life scenarios, intrusion detection systems are widely used in banking networks, healthcare systems, cloud platforms, corporate infrastructures, and government organizations to monitor traffic behavior, detect cyber-attacks early, and maintain secure and reliable communication systems.

- ## Steps of Implementation:

The implementation of the proposed Network Intrusion Detection System follows a structured Machine Learning pipeline to identify malicious network activities efficiently.

1.Dataset Simulation:

A realistic network traffic dataset is created containing multiple raw features such as IP Address, MAC Address, Source Host, Packet Size, Protocol Type, Bytes Sent, Bytes Received, Traffic Flag, Connection Duration, and Error Rate. The dataset intentionally includes missing values, noise, and irregular patterns to simulate real-world network conditions.

2.Data Organisation and Preprocessing:

The generated data is structured into a tabular dataset and labeled as Normal or Attack traffic to enable supervised learning. Data preprocessing techniques are applied, including handling missing values, detecting outliers, encoding categorical features, and performing exploratory data analysis.

3.Feature Scaling:

StandardScaler is applied to normalize all feature values so that each parameter contributes equally during distance calculation in the KNN algorithm.

4.Dataset Splitting:

The dataset is divided into training (80%) and testing (20%) sets to evaluate model performance on unseen data.

5.Optimal K Selection (Elbow Method):

Different values of K are tested, and an Elbow Plot is used to determine the optimal number of neighbors for accurate classification.

6.Model Training:

The K Nearest Neighbors (KNN) classifier is trained using the scaled training dataset to learn traffic patterns.

7.Prediction and Classification:

The trained model predicts whether incoming network traffic belongs to normal behavior or intrusion activity.

8.Performance Evaluation and Visualization:

Model performance is analyzed using Accuracy, Recall, Confusion Matrix, Elbow Plot, and Scatter Plot to interpret classification effectiveness.

- **Expected Modules/ Libraries to be used :**

Libraries and modules used are:

1. **NumPy:** Used for data manipulation and analysis.

2**. Pandas:** Provides support for numerical operations.

3. **Scikit-learn (sklearn):** Machine learning library providing algorithms and preprocessing tools. Modules used:

- **StandardScaler (sklearn.preprocessing)** – Normalizes feature values for KNN distance calculation.
- **train_test_split (sklearn.model_selection)** – Splits dataset into training and testing data.
- **KNeighborsClassifier (sklearn.neighbors)** – Implements KNN classification algorithm.
- **sklearn.metrics** – Computes accuracy, recall, and confusion matrix

4. **Matplotlib / Seaborn:** Used for visualization of results.

- **Expected Evaluation metrics and Plots to be used:**

1. **Accuracy** – Measures overall prediction correctness.
2. **Recall** – Measures ability to detect intrusion cases.
3. **Confusion Matrix** – Shows correct and incorrect classifications.
4. **Elbow Plot –** Helps select optimal K value for KNN.
5. **Scatter Plot –** Visualizes feature distribution to distinguish normal and attack traffic patterns.

This combination of Accuracy, Recall, Confusion Matrix, Elbow Plot, and Scatter Plot helps both quantitatively evaluate the model and visually interpret intrusion detection performance.

- **Required topics from the subject:**

1. **Data Preprocessing:** Preparing and scaling data before training improves model performance.
2. **Supervised Learning:** Used to classify network traffic into normal or attack categories.
3. **KNN Algorithm:** Classifies data based on similarity between network behavior patterns.
4. **Model Evaluation:** Helps measure reliability and accuracy of predictions.

- **Platform Required:**

1. **Jupyter Notebook** – For writing and running Python code to create the dataset and implement the ML model.
2. **Microsoft Excel / Google Sheets** – Useful for viewing, organizing, and inspecting the generated dataset.
3. **Microsoft Word / Google Docs –** Used for writing and formatting the project synopsis and documentation.

- **Books and Link Sources:**

1. Introduction to Machine Learning with Python – Scikit-learn Algorithms
2. Machine Learning – Oxford higher education
3. Scikit-learn – https://scikit-learn.org/stable/
4. Kaggle – https://www.kaggle.com/datasets
5. NumPy Documentation – https://numpy.org